# Markov Models Lab

*gallington*

*4/5/2021*

The first part of this document contains what we did last week. Scroll down to PART 2 for today's work.

## Intro

In this lab we will explore how to make projections of future landscape composition, based on past landscape change dynamics. You will need all of the files in the MarkovLab folder in the class Shared Drive.

First set your working directory to wherever you have stored your data files:

```
#setwd("./MarkovLab")
```

Let's explore the landscapes first. Load the classified imagery

```
pnw72<- raster("./images/pnw72.gif")
pnw84<- raster("./images/pnw84.gif")
pnw91<- raster("./images/pnw91.gif")
```

Let's also create a table of the class names for future reference in case we forget which class is which: Step 1. Put all the names of the classes into a vector that is just a string of text.

```
# we'll call it class.legend bc it's our legend of the names of the different growth classes.
class.legend <- c("Non-forest",
                  "ClearCut",
                  "EarlySeral",
                  "MidSeral",
                  "Mature",
                  "OldGrowth")
# then make another vector that's the matching class numbers, 0 through 5
class.numbers <- 0:5  # the colon means use zero through five
```

Step 2. Now we combine those two into one table, called a data frame, using the command `cbind()` which means to bind by columns. This will put each of the vectors into their own column.

```
pnw.classes<- as.data.frame(cbind(class.numbers, class.legend)) # this turns
# our vectors into a table
```

Step 3. Then we should probably name the columns in our table so we can remember what they are

```
names(pnw.classes)<- c("ClassNum", "ClassName")
# then view it
pnw.classes
```

```
##   ClassNum  ClassName
## 1        0 Non-forest
```

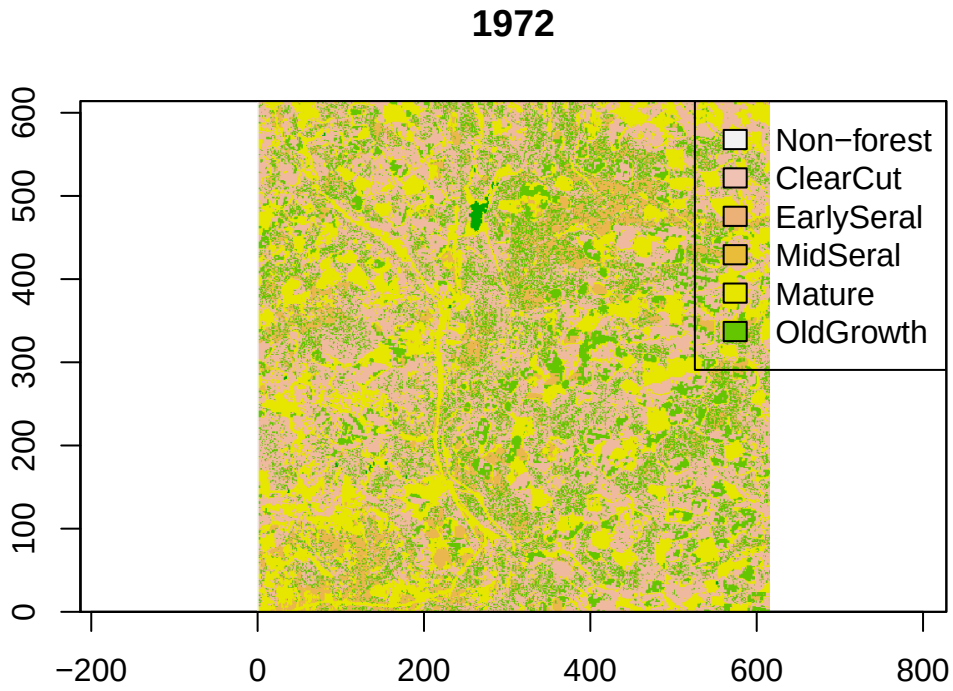
```
## 2         1   ClearCut
## 3         2 EarlySeral
## 4         3   MidSeral
## 5         4     Mature
## 6         5  OldGrowth
```

Now that we have that information in a dataframe, we can use that info to create a legend when we plot that rasters.

Plot 1972

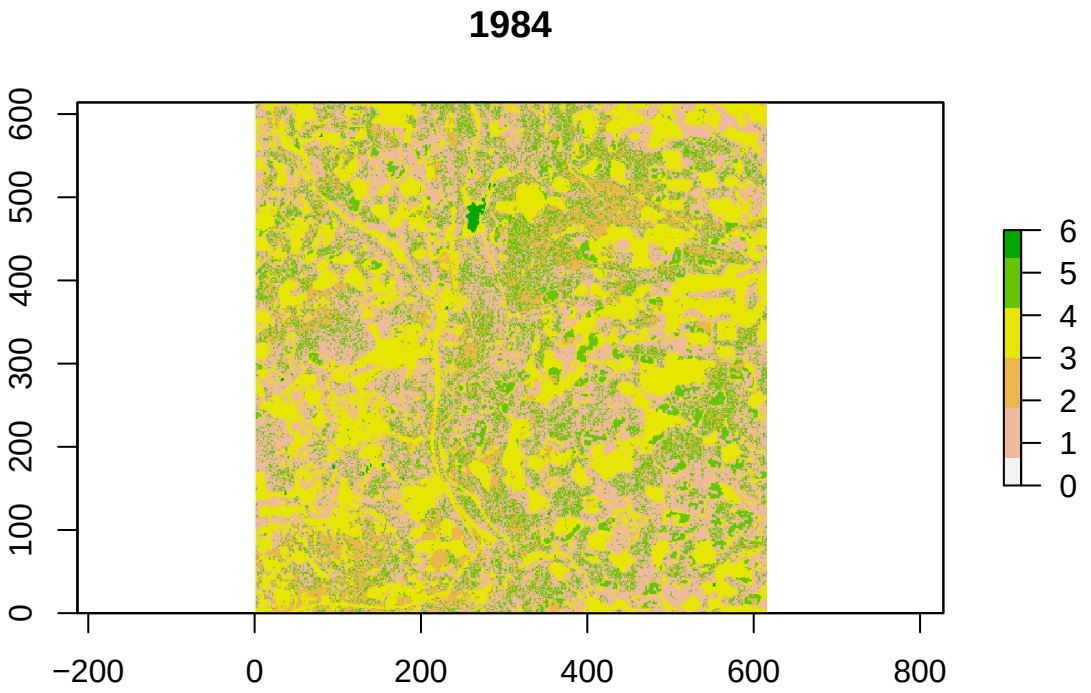
```
#landscapetools::show_landscape(pnw72)  # this is how we did it before

plot(pnw72, # but you can also do it this way
      main = "1972" ,
     col = (terrain.colors(n= 6, rev = TRUE)), # this uses a pre-existing color pallete,
     # and we have 6 classes, so we set the n to 6
     legend = FALSE)
legend("topright", legend = class.legend, fill = rev(terrain.colors(7)))
```

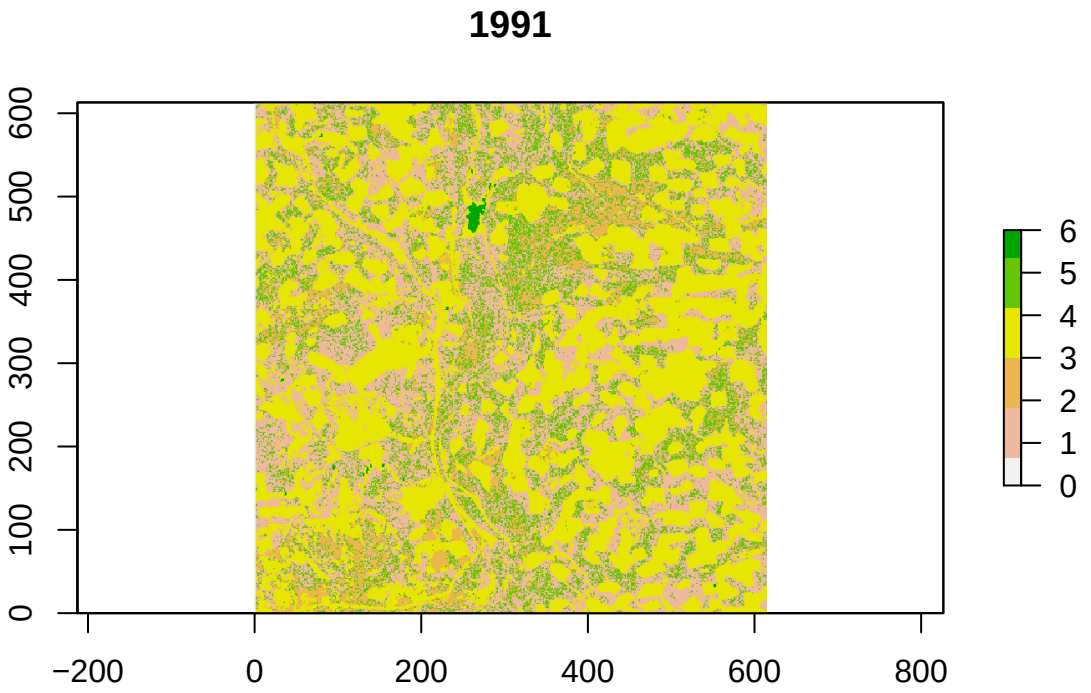



Now plot the 1984 and 1991 rasters:

```
plot(pnw84, # but you can also do it this way
      main = "1984" , col = rev(terrain.colors(6)))
```




```
plot(pnw91, # but you can also do it this way
      main = "1991" , col = rev(terrain.colors(6)))
```




## Modeling transitions over time

Now we can move on to the modeling work.

First load the sample data, which has already been generated for you, by casting 200 random points over the image, and then extracting the forest type at those locations. So, now we have a *sample* of the landscape, and from that we are going to draw some generalizations about the full landscape. This dataframe lists the observed class of 200 randomly cast points for each year. Load the data file and explore it:

```
# 200 sample points that have been examined and their classifications confirmed.
pnw.data <- read.csv("sample200.csv")
names(pnw.data)
```

```
## [1] "Sample" "X1972"  "X1984"  "X1991"
```

```
dim(pnw.data)
```

```
## [1] 200   4
```

```
  # also store the # of rows to it's own variable bc we will need
  # that later to calculate the proportions
sample.n <- dim(pnw.data)[[1]] # 1st dimension, nrow, = N
```

Determine the initial conditions in 1972 (the xt):

Now we'll calculate the tally and proportion vectors. You can generate the counts for each class with the `table()` function. We'll save that to a variable, and then use that, combined with the # rows variable we created above, to convert the tallies to proportions.

```
  # tally for 1972
t1972 <- table(as.factor(pnw.data$X1972))
  # proportions of each class in 1972
p1972 <- t1972/sample.n   # remember sample.n is what we created above
```

Let's use that to create a vector of the initial proportion of each class this is the xt that you'll need to multiply by the transition matrix

```
inits <- as.vector(p1972)  # representing the inital year of data.
# I called this inits in class so I'm keeping that here, but really
# if I were to do this again I'd just call it xt
xt <- as.vector(p1972) # since that's what it represents.
```

Now calculate the tallies & proportions for 1984 and 1991

```
  # tally for 1984
t1984 <- table(as.factor(pnw.data$X1984))
  # proportions of each class in 1984
p1984 <- t1984/sample.n
  # etc.
t1991 <- table(as.factor(pnw.data$X1991))
p1991 <- t1991/sample.n
```

## look at the data over time . . .

We want to create a plot to visualize the relative proportions of each class in each year. There are a few steps involved in bringing the data together and just so you understand what is happening, we'll take a look at what you created after each step, by calling the variable directly so you can see it.

```
  # first another vector with the labels for the three years
years <- c(1972, 1984, 1991)
years
```

```
## [1] 1972 1984 1991
```

Now we combine the proportions for each year that we created before, all into one. Remember, `rbind()` means to bind the data by row (to stick them on top of each other). We only want the proportions, not the labels, so we use `as.vector` to just get that info.

```
pnw.d2 <- rbind(as.vector(p1972), as.vector(p1984), as.vector(p1991))
pnw.d2  # this is essentially a matrix
```

```
##       [,1]  [,2]  [,3]  [,4]  [,5]
## [1,] 0.185 0.040 0.105 0.215 0.455
## [2,] 0.230 0.055 0.100 0.205 0.410
## [3,] 0.375 0.055 0.090 0.155 0.325
```

Now turn it into a dataframe(like a table) and add the years that we created above

```
pnw.d2 <- data.frame(cbind(years, pnw.d2))
pnw.d2  # same name as above, you've just changed the format to table
```

```
##   years    V2    V3    V4    V5    V6
## 1  1972 0.185 0.040 0.105 0.215 0.455
## 2  1984 0.230 0.055 0.100 0.205 0.410
## 3  1991 0.375 0.055 0.090 0.155 0.325
```

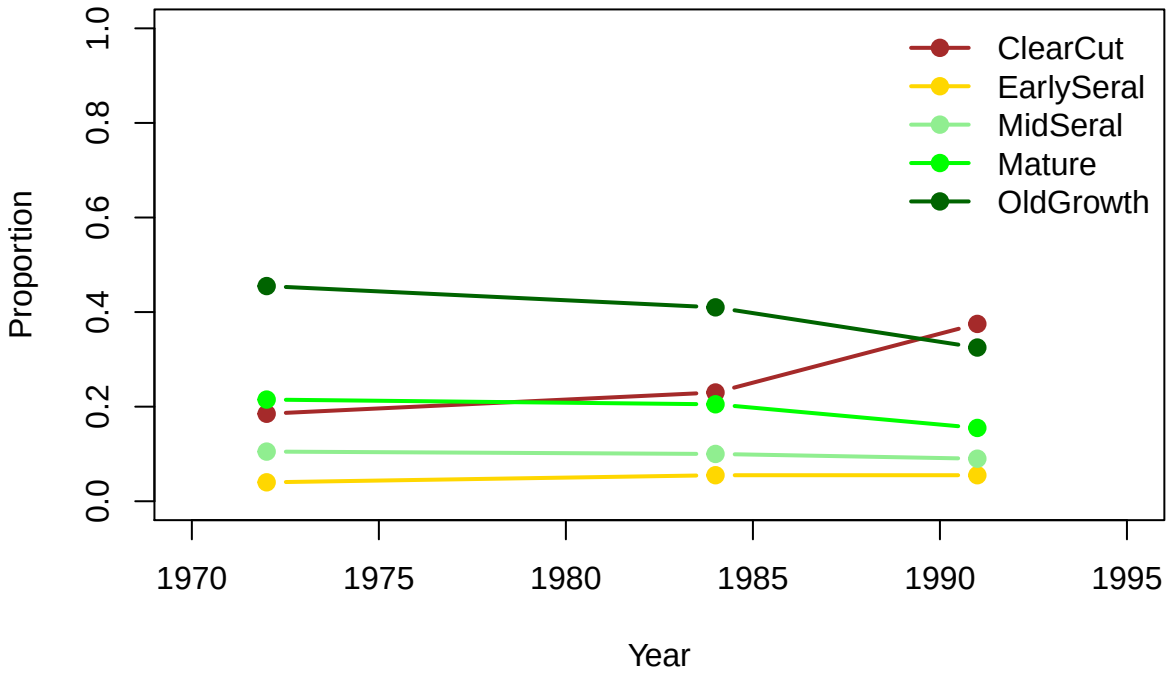
```
  # but the columns don't have names yet
  # name the columns: Year and class names
names(pnw.d2) <- c("Year",
                   "ClearCut",
                   "EarlySeral",
                   "MidSeral",
                   "Mature",
                   "OldGrowth")

# take a look at what you've made!
# these are the proportions observed in each year, from the sampled data
pnw.d2
```

```
##   Year ClearCut EarlySeral MidSeral Mature OldGrowth
## 1 1972    0.185      0.040    0.105  0.215     0.455
## 2 1984    0.230      0.055    0.100  0.205     0.410
## 3 1991    0.375      0.055    0.090  0.155     0.325
```

I know, that was a lot of steps, just to get a table of data that we can plot, but it's useful for later steps too. We'll just visualize it first. ## So, now let's plot the data: Remember, all of this information comes from the 200 sample points. Just copy and paste this whole thing into your script.

```
attach(pnw.d2)
  # plot the clearcut first
plot(Year, ClearCut, xlim=c(1970,1995), xlab="Year", ylim=c(0,1),
     ylab="Proportion", type="b", pch=19, col="brown", lwd=2)
  # now add lines for the other growth stages:
lines(Year,EarlySeral,type="b",pch=19,col="gold",lwd=2)
lines(Year,MidSeral,type="b",pch=19,col="light green",lwd=2)
lines(Year,Mature,type="b",pch=19,col="green",lwd=2)
lines(Year,OldGrowth,type="b",pch=19,col="dark green",lwd=2)
# now add a legend
legend(x="topright",
       legend=c("ClearCut","EarlySeral","MidSeral","Mature","OldGrowth"),
       lwd=2,
       pch=19,
       col=c("brown","gold","light green","green","dark green"),
       bty="n")
```




## Now we'll use that data to generate a transition matrix:

Use this data to complete Tables 1 & 2 in the excel spreadsheet.

##First make the raw tally matrixx *NOTE:* R generates the matrices with t1 on the rows and t2 on the columns

```
# build the raw tally matrix  ...
t.72.84 <- table(pnw.data$X1972,pnw.data$X1984)
t.72.84 # Enter these values into table 1.
```

```
##
##      1  2  3  4  5
##   1 33  4  0  0  0
##   2  0  7  1  0  0
##   3  0  0 19  2  0
##   4  3  0  0 39  1
##   5 10  0  0  0 81
```

### Now make the transition matrix ( $P$ )

Divide each element in the raw tally matrix by it's row total to get the transition matrix (a matrix of probabilities of transitioning from type i to j). For instance, for Clearcut (class 1), the row total would be `33 + 4 + 0 + 0 + 0 = 37` And the probability of a cell staying a clearcut would be `33/37 = 0.89` which

is what you would enter into the first cell of the transition matrix. Enter your values into the cells in Table 2. You can just do it by hand, or make a function in the excel sheet.

In the final step for now, we will check your work by using a function that has already been created to generate the transition matrix.

```
# relativize by row sum with a function that has already been created:
source("rel.R")  # This loads a function rel() contained in a diff script
  # this uses that function
  # need to specify which transition matrix you want,
  # and whether you want to cal by row or column
tr.72.84 <- rel(t.72.84, byrow=TRUE, bycol=FALSE)
tr.72.84   # this is the transition matrix (P)
```

```
##
##              1          2          3          4          5
##   1 0.89189189 0.10810811 0.00000000 0.00000000 0.00000000
##   2 0.00000000 0.87500000 0.12500000 0.00000000 0.00000000
##   3 0.00000000 0.00000000 0.90476190 0.09523810 0.00000000
##   4 0.06976744 0.00000000 0.00000000 0.90697674 0.02325581
##   5 0.10989011 0.00000000 0.00000000 0.00000000 0.89010989
```

**The transition matrix that we calculated above this is for a 12-yr time step, but we need annual steps for our projections. . . unless we only want to make projections in 12-yr steps. . .**

## Get the annual transition probabilities so we can have a true P matrix

Convert the transition matrix to an annual timestep. This is partly cosmetic (transient dynamics will look smoother), but will also make it possible for us to use the info from the 12-year timestep of the first period to ijfer changes in the 7-year timestep of 1984–1991. To convert the transition probability matrix $P$ to an annual timestep, do the following: a. Divide each of the off-diagonal elements by 12. b. Adjust the diagonal elements to be 1 - the total of the off-diagonals. (see also the lab chapter pdf, which describes how to do this) In other words, all rows must sum to 1.0. **Enter the results in Table 3.**

We can also check your results with another script that does those calculations for you. (But I wanted you to do them by hand so that you understand what's going on)

```
  # so load this script as well
source("fix_tm.R")
  # and then use this function below, which generates
  # annual probability of transition
  # you need to specify which transition matrix you want
  # and also how many years
  # so we want the one based on 1972-1984, which covers 12 years:
P <- fix.steps(tr.72.84, 12)
```

Let's look at that annual transition matrix (P) *Remeber that starting condition is in the rows*

```
P
```

```
##
##               1           2           3           4           5
##   1 0.990990991 0.009009009 0.000000000 0.000000000 0.000000000
```

```
##   2 0.000000000 0.989583333 0.010416667 0.000000000 0.000000000
##   3 0.000000000 0.000000000 0.992063492 0.007936508 0.000000000
##   4 0.005813953 0.000000000 0.000000000 0.992248062 0.001937984
##   5 0.009157509 0.000000000 0.000000000 0.000000000 0.990842491
```

Check that these values match what you calculated by hand. **Question:**
Let's remembeber why we're doing any of this. We want to learn about the rates of harvest and regrowth in the logging of old growth forests. What was the annual harvest rate of old growth forests between 1972 and 1984? (check pnw.classes if you forget which class is which) Please HIGHLIGHT the cell in Table 3 that represents the harvest rate of old growth forest.

Now you're done! Please upload your excel file and your R script to Blackboard. ***

# PART TWO:

First, run everything from last week, so that you have everything ready to go here and can reference those data.

For today – you can go ahead and copy and paste the code for PLOTTING from the sections below right into your script. Don't worry about trying to retype the plotting commands. That's too easy to get typos, etc! I tried to annotate everything so that you can go back and make sense of each step later.

We can calculate P for the other time step (1984-1991), for comparison:

```
t.84.91 <- table(pnw.data$X1984,pnw.data$X1991)
t.84.91
```

```
##
##      1  2  3  4  5
##   1 46  0  0  0  0
##   2  0 11  0  0  0
##   3  2  0 18  0  0
##   4 10  0  0 31  0
##   5 17  0  0  0 65
```

```
tr.84.91 <- rel(t.84.91, byrow=TRUE, bycol=FALSE)

P.84.91 <- fix.steps(tr.84.91, 7)
  # annual probability of transition to other class
P.84.91
```

```
##
##              1          2          3          4          5
##   1 1.00000000 0.00000000 0.00000000 0.00000000 0.00000000
##   2 0.00000000 1.00000000 0.00000000 0.00000000 0.00000000
##   3 0.01428571 0.00000000 0.98571429 0.00000000 0.00000000
##   4 0.03484321 0.00000000 0.00000000 0.96515679 0.00000000
##   5 0.02961672 0.00000000 0.00000000 0.00000000 0.97038328
```

**QUESTION:** What was the annual harvest rate of old growth forest from 1984-1991? Was this higher or lower than the 70s-early 80s?

## Projections

Now that we have this information, we can use it to project in to the future, so that we can ask questions like: If the annaual harvest rate continued at ~3%, what would the composition of the landscape be like in a few decades? Or a few hundred years?

```
  # set a variable that specifies how many land cover types.
  # remembber, you created that `inits` object earlier. If
  # you can't remember what is in it, call it in the console
  # or scroll up to refresh your memory.
  # here use `length()` to ask how many items are in the vector:
ntypes <- length(inits) # 5 types, in this case

# create some empty data frames for the output
ntimes <- 999
t <- vector(length=ntimes)
transient <- matrix(nrow=ntimes, ncol=ntypes)
```

Let's stop and explore what we just created, so we know what we've done: (we created some empty objects)

```
str(t)  # str() is a way to examine the structure of the object
```

```
##  logi [1:999] FALSE FALSE FALSE FALSE FALSE FALSE ...
```

```
  # you could also just call it directly in the console, but it if
  # it's large, like `transient`, that can be awkward.
str(transient)
```

```
##  logi [1:999, 1:5] NA NA NA NA NA NA ...
```

Now we need to specify a few more things:

```
tmp <- inits # copy these to simplify the projections

t[1] <- 1972   # set the start values -- start year &
transient[1,] <- tmp    # start proportions

t1 <- 1972   # start year
```

And finally, here's a function to actually do the projections: It loads the numbers in to our empty objects that we created above.

```
# the actual projection:
for (i in 1:ntimes) {
    t[i]<-t1+i   # update each NA in t by adding one to the previous
    transient[i,]<-tmp %*% P # matrix multiplication in R lingo
    tmp<-transient[i,] # updates the vector of proportions each year
}                    # by multiplying the previous yr by P
# now check out transient
head(transient)
```

```
##           [,1]       [,2]      [,3]      [,4]      [,5]
## [1,] 0.1887500 0.04125000 0.1045833 0.2141667 0.4512500
## [2,] 0.1924270 0.04252076 0.1041830 0.2133365 0.4475327
## [3,] 0.1960321 0.04381142 0.1037991 0.2125096 0.4438479
## [4,] 0.1995661 0.04512110 0.1034316 0.2116860 0.4401952
## [5,] 0.2030300 0.04644898 0.1030808 0.2108659 0.4365743
## [6,] 0.2064248 0.04779424 0.1027465 0.2100494 0.4329851
```

```
# some cosmetics ...
# add a "year" column for convenience-- use the vector
# t that you created above:
out <- data.frame(cbind(t, transient))
head(out)
```

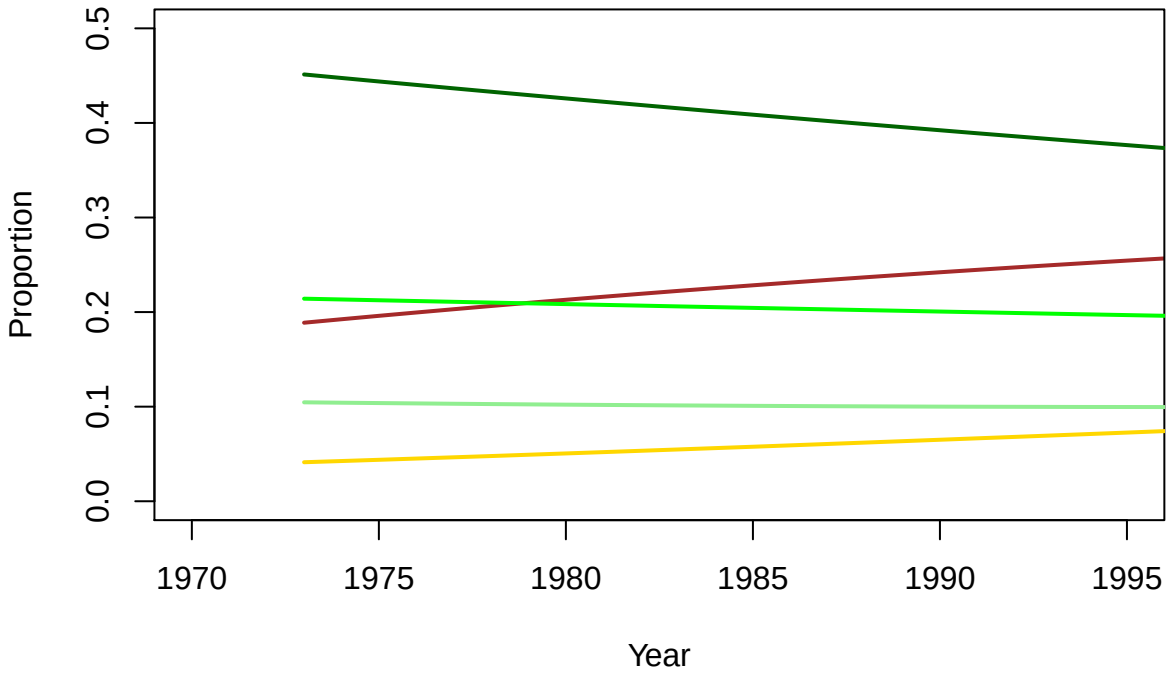
```
##      t        V2         V3        V4        V5        V6
## 1 1973 0.1887500 0.04125000 0.1045833 0.2141667 0.4512500
## 2 1974 0.1924270 0.04252076 0.1041830 0.2133365 0.4475327
## 3 1975 0.1960321 0.04381142 0.1037991 0.2125096 0.4438479
## 4 1976 0.1995661 0.04512110 0.1034316 0.2116860 0.4401952
## 5 1977 0.2030300 0.04644898 0.1030808 0.2108659 0.4365743
## 6 1978 0.2064248 0.04779424 0.1027465 0.2100494 0.4329851
```

```
# create labels for the output columns:
col.labels <- c("Year","ClearCut","EarlySeral","MidSeral","Mature","OldGrowth")
names(out)<-col.labels

# dump the output to a CSV, in case we want to open it in Excel:
write.csv(out,"markov_out.csv")
```

**Model Verification** Plot the output, first for just the initial few years:

```
plot(out[,1], # specifies use first column of data for x-axis,
     out[,2],           # second col of data for y-axis
     type="l",          # graph type is a line
     col="brown",       # line color is brown
     lwd=2,             # line width  = 2
     xlim=c(1970,1995), # x-axis limits
     ylim=c(0,0.5),     # y-axis limits
     xlab="Year",       # x-axis label
     ylab="Proportion") # y-axis label
  # these add additional lines to the same plot:
lines(out[,1],out[,3],type="l", col="gold", lwd=2)
lines(out[,1],out[,4],type="l", col="light green", lwd=2)
lines(out[,1],out[,5],type="l", col="green", lwd=2)
lines(out[,1],out[,6],type="l", col="dark green", lwd=2)
```



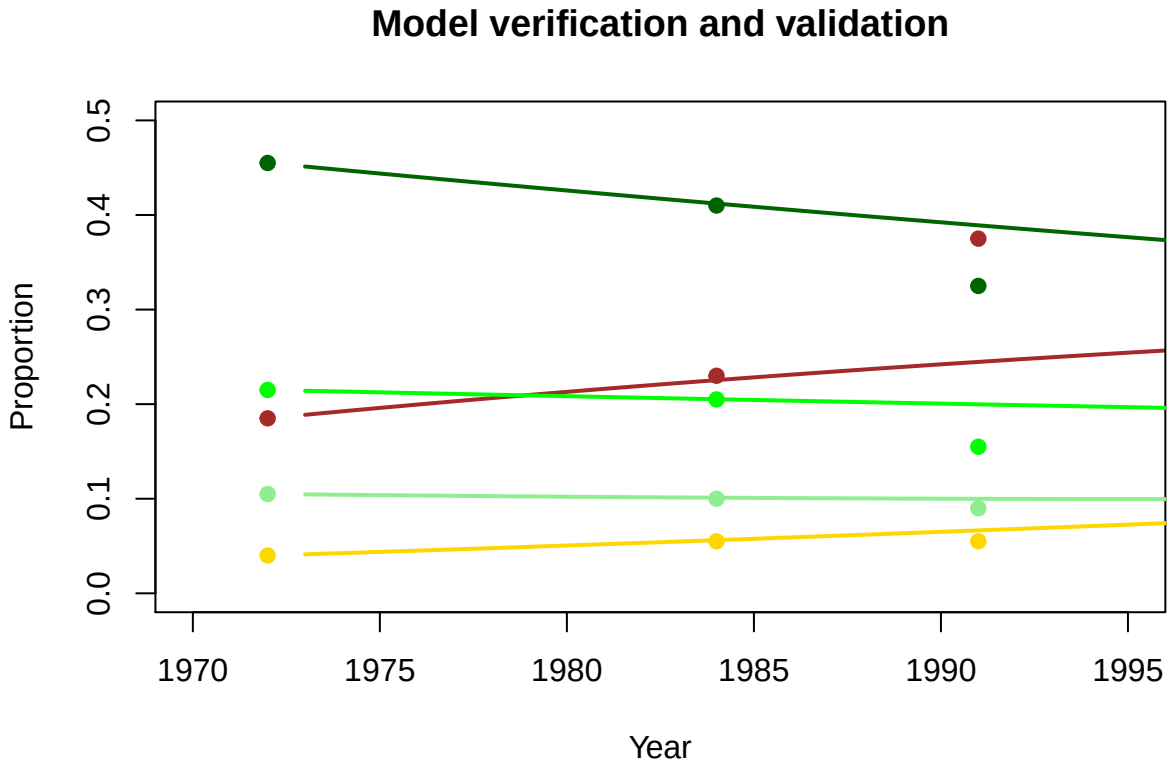

Now plot the actual data on top of the model projections: How did we do at replicating the actual observed values?

*The lines are the projected data, the dots are the observed (sampled) data.*

```
# same as above:
plot(out[,1],out[,2],type="l",col="brown", lwd=2,
     # pnly plotting the projections out to 1995 for now
     xlim=c(1970,1995),ylim=c(0,0.5),
     xlab="Year",ylab="Proportion")
lines(out[,1],out[,3],type="l", col="gold", lwd=2)
lines(out[,1],out[,4],type="l", col="light green", lwd=2)
lines(out[,1],out[,5],type="l", col="green", lwd=2)
lines(out[,1],out[,6],type="l", col="dark green", lwd=2)

# put the actual data points on top of the model projections:
points(rep(years[1],ntypes),
       p1972, pch=19,
       col=c("brown","gold","light green","green","dark green"))
points(rep(years[2],ntypes),
       p1984, pch=19,
       col=c("brown","gold","light green","green","dark green"))
points(rep(years[3],ntypes),
       p1991, pch=19,
       col=c("brown","gold","light green","green","dark green"))
  # I'm commenting out the legend so it doesn't cover up the
  # lines, but if you want to see it, just remove the hash
```

```
  # from teh start of this line below.
#legend(x="topright",legend=names(out[2:6]), lwd=2,
#     col=c("brown","gold","light green","green","dark green"),
#     pch=19)
title("Model verification and validation")
```



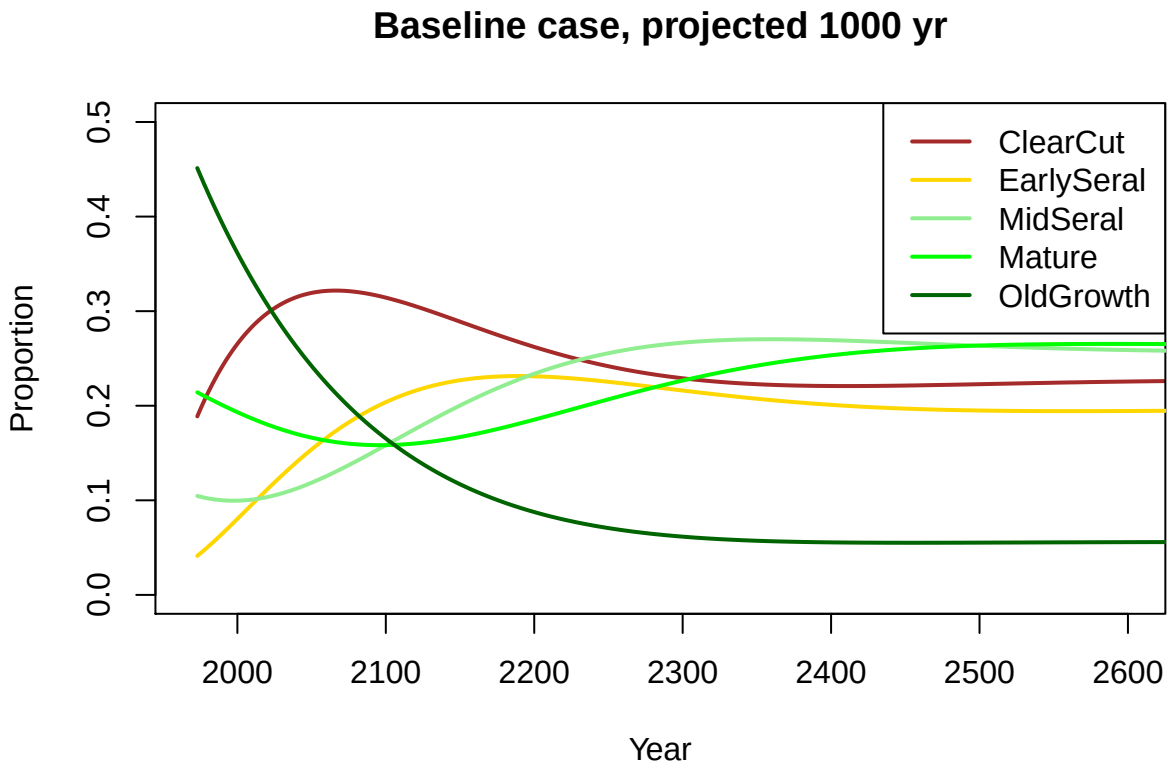

**Question:**
Does the model projection match the observed values? If not, can you suggest some reasons why?

### Model Projection

Plot the entire simulation, so we can see the projected trends in to the future:

```
plot(out[,1],out[,2],type="l",col="brown", lwd=2,
    # now plotting all of the projected data
     xlim=c(1970,2600),ylim=c(0,0.5),
     xlab="Year",ylab="Proportion")
lines(out[,1],out[,3],type="l", col="gold", lwd=2)
lines(out[,1],out[,4],type="l", col="light green", lwd=2)
lines(out[,1],out[,5],type="l", col="green", lwd=2)
lines(out[,1],out[,6],type="l", col="dark green", lwd=2)
legend(x="topright",
       legend=names(out[2:6]),
       col=c("brown","gold","light green","green","dark green"),
```

```
       lwd=2)
title("Baseline case, projected 1000 yr")
```




**Question:**
Does the model ever reach a steady state? (no transitions)
At approximately what year does the amount of remaining Old Growth Forest fall below 10%?

**Take home:**

Try adjusting the harvest rates, by cutting them in half . . . What happens to the prevalence of old growth & mature forests?

We'll adjust the rates by changing values in the transition probabilty matrix ($P$). We'll update the values directly by specifying the location of the value using it's index. We discussed indexing briefly last week. Essentially, if you have a dataframe (table) or matrix, each cell has a location specified by it's row and column location. we use brackets, and list row then column. As in: [row#, col#]. So, the first row in the second column would be indexed with [1,2]. You can use the index to just call a value, i.e. `P[3,4]` or to actually do somethign to that value, like double it: `P[3,4]/2` or to replace it with something else: `P[3,4] <- 0.45`

[rowposition , columnposition]

```
P2 <- P # make a  copy of P so you can update values
# look at it to remember what values live where in the matrix:
P2
```

```
##
##              1           2           3           4           5
##   1 0.990990991 0.009009009 0.000000000 0.000000000 0.000000000
##   2 0.000000000 0.989583333 0.010416667 0.000000000 0.000000000
##   3 0.000000000 0.000000000 0.992063492 0.007936508 0.000000000
##   4 0.005813953 0.000000000 0.000000000 0.992248062 0.001937984
##   5 0.009157509 0.000000000 0.000000000 0.000000000 0.990842491
```

```
  # row 4, column 1:
P[4,1]   # this is the cell for transition from mature forest to clear cut.
```

```
## [1] 0.005813953
```

```
h41 <- P[4,1]/2  # divide it in half
P2[4,1] <- h41   # and replace it in the matrix
P2[4,4] <- P[4,4]+h41  # add the remaining to stay mature forest. you can
        # just add the value, bc you divided it in half originally.
  # row 5, column 1:
P[5,1]    # this is the cell for transition from old growth forest to clear cut.
```

```
## [1] 0.009157509
```

```
h51 <- P[5,1]/2   # divide it in half
P2[5,1] <- h51   # and replace it in the matrix
P2[5,5] <- P2[5,5]+h51  # add the remaining to stay old growth forest
```

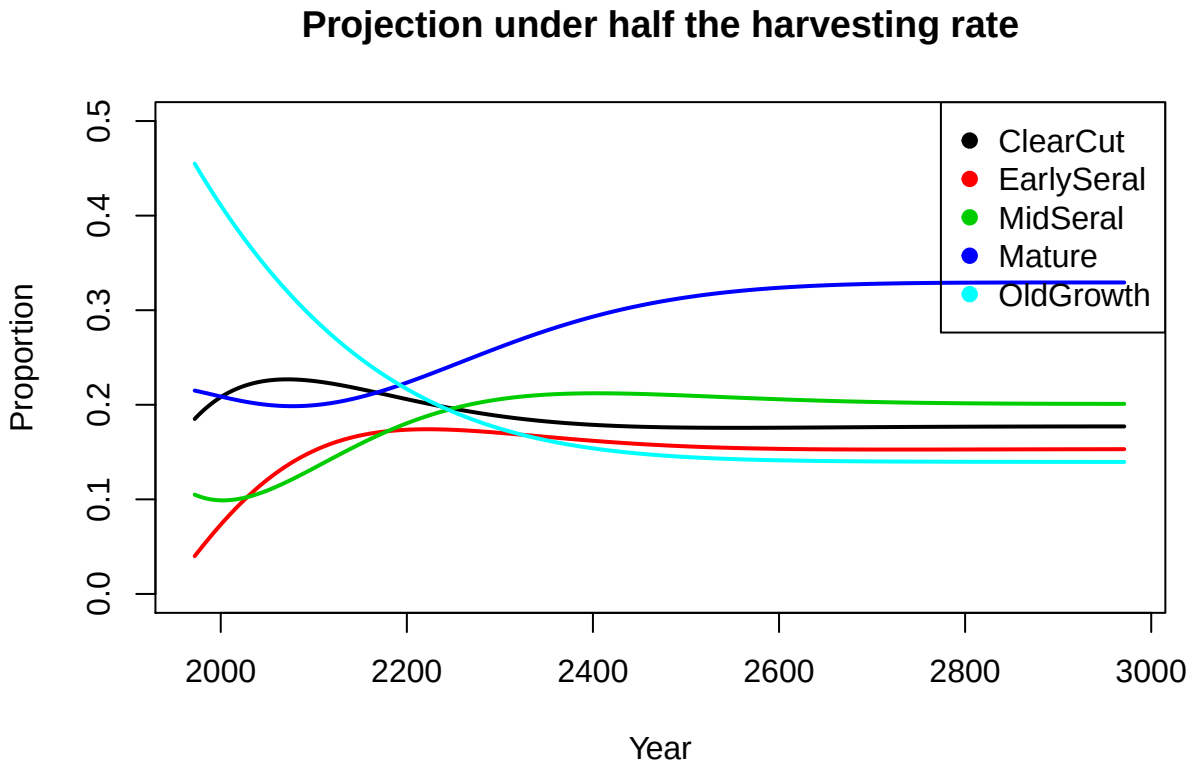
**RERUN THE PROJECTION W THESE NEW RATES**

```
  # make a new empty matrix to store the values in:
trans2 <- matrix(nrow=ntimes, ncol=ntypes)
  # specify the start year:
t1<-1972
  # specify the initial conditions:
trans2[1,]<-inits
  # create a copy of inits to start off the projections:
tmp<-inits
# the projection:  (same idea as last time)
for (i in 1:ntimes) {
    t[i]<-t1+i
    trans2[i,]<-tmp %*% P2 # matrix multiplication in R lingo
    tmp<-trans2[i,]
}
out2 <- data.frame(cbind(t, trans2)) # add the year
out2<- rbind(c(t1,inits),out2) # add the inital year
#names(out2)<-c("time",paste("C",as.character(1:ntypes),sep=""))
names(out2)<- col.labels
```

**plot the new simulation:**

This shows another away to specify which column of data you want to plot, based on the index location.

```r
plot(out2[,1],out2[,2],
     type="l",col=1, lwd=2,
     xlim=c(1970,2975),ylim=c(0,0.5),
     xlab="Year",ylab="Proportion")
lines(out2[,1],out2[,3],type="l", col=2, lwd=2)
lines(out2[,1],out2[,4],type="l", col=3, lwd=2)
lines(out2[,1],out2[,5],type="l", col=4, lwd=2)
lines(out2[,1],out2[,6],type="l", col=5, lwd=2)
legend(x="topright",legend=names(out[2:6]),col=1:5,pch=19)
title("Projection under half the harvesting rate")
```




**QUESTION** Can you find a harvest rate that will keep old growth forests above 30%? (reuse the code above– just copy and paste it in your script and make new changes) What other factors would have to change in order to improve the outcome?

How realistic is it for us to use the annual harvest rate from the 1970s to project forward 1,000 years?

PLEASE POST YOUR UPDATED SCRIPT AND SIMPLY INCLUDE YOUR ANSWERS IN THE SCRIPT BY COMMENTING OUT (#) THE LINES WITH YOUR ANSWERS.